

Conteúdos Abordados

- **Ordenação Externa (Intercalação / Merging):**
 - * Introdução: Necessidade e cenários.
 - * Modelo Geral do Sort-Merge Externo:
 - Fase 1: Criação de "runs" iniciais.
 - Fase 2: Intercalação (merge) das "runs".
 - * Intercalação Balanceada de k-vias (k-way merge):
 - Conceito, buffers de entrada/saída, seleção do elemento.
 - * Estratégias Avançadas (menção conceitual): Intercalação Polifásica, Reposição com Seleção.
 - * Análise de Desempenho: Foco em passadas e I/O.
- **Tabelas Hash (Hashing):**
 - * Conceito Fundamental e Componentes.
 - * Funções Hash (Funções de Espalhamento):
 - Propriedades desejáveis.
 - Métodos Comuns (Divisão, Multiplicação).
 - Tratamento de chaves não numéricas.
 - * Tratamento de Colisões:
 - Definição de colisão.
 - Endereçamento Aberto (Open Addressing):
 - ◊ Sondagem Linear, Quadrática, Hashing Duplo.
 - ◊ Remoção e marcadores "DELETED".
 - Encadeamento Separado (Separate Chaining):
 - ◊ Uso de listas encadeadas (ou outras estruturas).
 - * Fator de Carga (α) e seu impacto.
 - * Redimensionamento (Rehashing): Necessidade, processo, custo.
 - * Aplicações, Análise de Desempenho, Vantagens e Desvantagens.

Conceitos, Algoritmos e Estratégias Detalhadas

- **Ordenação Externa (Intercalação / External Sort-Merge):** *Para ordenar conjuntos de dados que não cabem na memória principal, usando armazenamento secundário.*
 - **Princípio Geral (Duas Fases):**
 - 1) **Fase 1: Criação de "Runs" Iniciais Ordenadas:**
 - Ler blocos de dados do arquivo original para a memória.
 - Ordenar cada bloco internamente (ex: Quicksort, Heapsort).
 - Escrever as sequências ordenadas ("runs") em arquivos temporários no disco.

- *Otimização*: Usar técnicas como "Reposição com Seleção" (Selection Replacement) para gerar "runs" iniciais com tamanho médio maior que a memória disponível.

2) Fase 2: Intercalação (Merge) das "Runs":

- Combinar progressivamente as "runs" ordenadas em "runs" maiores até restar uma única "run" final ordenada.

– Intercalação de k-vias (k-way merge):

Entrada: k "runs" iniciais ordenadas $(R_1, \dots, R_k)$, k buffers de entrada $(B_1, \dots, B_k)$, 1 buffer de saída (B_{out}) .

- 1: Carregar o primeiro bloco de cada R_j para o respectivo B_j .
- 2: **while** pelo menos um buffer de entrada não está vazio e não marcou fim de "run" **do**
- 3: Encontrar o menor elemento e_{min} entre os primeiros elementos disponíveis em $B_1, \dots, B_k$.
- 4: Mover e_{min} para B_{out} .
- 5: **if** B_{out} está cheio **then**
- 6: Escrever B_{out} para a "run" de saída. Limpar B_{out} .
- 7: Avançar para o próximo elemento na "run" de onde e_{min} foi retirado.
- 8: **if** o buffer de entrada correspondente (B_j) esvaziar E a "run" R_j não terminou **then**
- 9: Carregar o próximo bloco de R_j para B_j .
- 10: **if** B_{out} não está vazio **then**
- 11: Escrever B_{out} para a "run" de saída.

Nota: Para k grande, um min-heap pode ser usado para encontrar e_{min} eficientemente.

– Considerações de Desempenho:

- Principal custo: operações de I/O (leitura/escrita em disco).
- Número de passadas de intercalação: $\approx \lceil \log_k N_r \rceil$, onde N_r é o número de "runs" iniciais.
- Custo de I/O por passada: Leitura de todas as "runs" da passada anterior e escrita das novas "runs" combinadas.

• Tabelas Hash (Hashing): Mapeamento eficiente de chaves para posições em um array (tabela hash).

- **Conceito Central:** $indice = h(chave)$, $0 \leq indice < m$ (tamanho da tabela).

– Funções Hash Comuns:

- **Método da Divisão:** $h(k) = k \pmod{m}$. *Recomendação:* m ser um número primo.
- **Método da Multiplicação:** $h(k) = \lfloor m \cdot (kA - \lfloor kA \rfloor) \rfloor$, onde $0 < A < 1$ (ex: $A \approx (\sqrt{5} - 1)/2$).
- **Hashing para Strings:** Considerar cada caractere como um número (ex: ASCII) e combiná-los polinomialmente, aplicando $\pmod{m}$ ao final. Ex: $h(S) = (\sum_{i=0}^{|S|-1} S[i] \cdot p^i) \pmod{m}$, para um primo p .

- **Tratamento de Colisões:** ($h(k_1) = h(k_2)$ para $k_1 \neq k_2$)
 - **Encadeamento Separado (Separate Chaining):** *Estrutura:* Cada $T[j]$ aponta para uma lista (ou outra estrutura) de chaves k tais que $h(k) = j$.
 - 1: **function** INSEREENCADEADO($T, chave, valor$)
 - 2: $idx \leftarrow h(chave)$
 - 3: Inserir ($chave, valor$) na lista $T[idx]$.
 - 4: **function** BUSCAENCADEADO($T, chave$)
 - 5: $idx \leftarrow h(chave)$
 - 6: Percorrer lista $T[idx]$ procurando por $chave$.
 - 7: **return** valor associado ou "não encontrado".

Fator de Carga $\alpha = n/m$ (n^o chaves / tamanho tabela): Pode ser > 1 . *Desempenho esperado:* Busca/Remoção $\approx O(1 + \alpha)$.

- **Endereçamento Aberto (Open Addressing):** *Estrutura:* Todos elementos na própria tabela. Se $h(k)$ está ocupado, sonda-se $h_0(k), h_1(k), \dots$. *Fator de Carga* $\alpha = n/m$: Deve ser $\alpha < 1$. *Sequência de Sondagem:* $h_i(k)$ para $i = 0, 1, 2, \dots$.

- 1: **function** INSEREENDERABERTO($T, chave, valor$)
- 2: **for** $i \leftarrow 0$ to $m - 1$ **do**
- 3: $idx \leftarrow h_i(chave, m)$ ▷ Sondagem i
- 4: **if** $T[idx]$ está VAZIO ou $T[idx] == DELETED$ **then**
- 5: $T[idx] \leftarrow (chave, valor)$
- 6: **return** SUCESSO
- 7: **return** FALHA (tabela cheia)
- 8: **function** BUSCAENDERABERTO($T, chave$)
- 9: **for** $i \leftarrow 0$ to $m - 1$ **do**
- 10: $idx \leftarrow h_i(chave, m)$
- 11: **if** $T[idx]$ está VAZIO **then**
- 12: **return** NÃO ENCONTRADO
- 13: **if** $T[idx].chave == chave$ **and** $T[idx] \neq DELETED$ **then**
- 14: **return** $T[idx].valor$
- 15: **return** NÃO ENCONTRADO

Tipos de Sondagem: **1. Linear:** $h_i(k) = (h'(k) + i) \pmod{m}$. Causa clustering primário. **2. Quadrática:** $h_i(k) = (h'(k) + c_1 i + c_2 i^2) \pmod{m}$. Causa clustering secundário. **3. Duplo Hashing:** $h_i(k) = (h_1(k) + i \cdot h_2(k)) \pmod{m}$. $h_2(k) \neq 0$ e idealmente primo com m . Reduz clustering. *Remoção:* Usar marcador "DELETED" para não quebrar sequências de sondagem. *Desempenho:* Fortemente dependente de α . Degrada rapidamente quando α se aproxima de 1.

- **Análise de Desempenho (Tempo Médio Esperado sob Hashing Uniforme Simples):**
 - **Encadeamento Separado:** Busca/Inserção/Remoção: $O(1 + \alpha)$.
 - **Endereçamento Aberto (Busca Malsucedida/Inserção):** Sondagem Linear: $\approx O(1/(1 - \alpha)^2)$. Sondagem Quadrática/Duplo Hashing: $\approx O(1/(1 - \alpha))$.